ROAD DAMAGE DETECTION

Using Convolutional Neural Network (CNN)

Professional Project Documentation

Project Type	Deep Learning / Computer Vision
Application	Road Damage Image Classification
Model	Convolutional Neural Network (CNN)
Classes	Plain Road / Pothole
Platform	Google Colab
Framework	TensorFlow / Keras
Author	Shafaat Ullah
Project Focus	Image Classification and Road Damage Detection

This document describes the complete workflow used to develop a CNN-based road damage classification system, from dataset preparation and image preprocessing to model training, evaluation, prediction, and model saving.

1. Abstract

Road surface damage such as potholes can create safety risks, increase vehicle maintenance costs, and require timely infrastructure maintenance. Manual road inspection is often time-consuming and difficult to scale. This project develops a deep-learning image classification system that identifies road images as either **Plain** or **Pothole** using a Convolutional Neural Network (CNN).

The project was implemented in Google Colab using TensorFlow and Keras. Images were resized to 224×224 pixels and normalized before being passed to the CNN. Data augmentation was used to improve generalization. The improved model uses Batch Normalization, Dropout, Global Average Pooling, Early Stopping, learning rate reduction, and model checkpointing.

2. Project Objectives

- Build an automated image-based road damage classification system.
- Classify road images into Plain Road and Pothole categories.
- Apply image preprocessing and augmentation to improve model generalization.
- Develop a CNN suitable for binary image classification.
- Evaluate the model using accuracy, loss, classification report, and confusion matrix.
- Save the trained model for future prediction and deployment.

3. Tools and Technologies

Category	Technology
Programming Language	Python
Development Platform	Google Colab
Deep Learning Framework	TensorFlow / Keras
Numerical Computing	NumPy
Data Visualization	Matplotlib / Seaborn
Machine Learning Evaluation	Scikit-learn
Model Type	Convolutional Neural Network
Image Size	224 × 224 × 3

4. Dataset Description

The project uses a road image dataset organized into two categories: **Plain** and **Pothole**. The dataset is suitable for binary image classification. The training images are loaded using Keras *ImageDataGenerator*, with 20% of the training directory reserved for validation.

Class 0 / Class 1: The exact numeric class assignment is determined automatically by Keras from the directory names, so the model does not depend on a manually hard-coded label order.

Dataset preparation workflow:

- Download and extract the dataset in Google Colab.
- Automatically identify the dataset directories.

- Detect the available class folders.
- Load images using Keras image generators.
- Resize images to 224×224 pixels.
- Normalize pixel values from 0–255 to 0–1.
- Use 80% of the training data for training and 20% for validation.

5. Image Preprocessing and Augmentation

Image preprocessing converts raw images into a consistent numerical format that can be processed efficiently by the neural network. Each image is resized to 224×224 pixels and rescaled using 1/255.

Training augmentation includes:

- Rotation up to 15 degrees
- Width and height shifting
- Shearing
- Zooming
- Horizontal flipping
- Nearest-neighbor filling for transformed areas

Validation images are only rescaled and are not augmented. This provides a more realistic measurement of how the model performs on unseen images.

6. CNN Architecture

Layer / Block	Configuration	Purpose
Input	224×224×3	Accept RGB road images
Block 1	Conv2D 32 + BatchNorm + MaxPool + Dropout 0.20	Low-level features
Block 2	Conv2D 64 + BatchNorm + MaxPool + Dropout 0.25	Intermediate features
Block 3	Conv2D 128 + BatchNorm + MaxPool + Dropout 0.30	Higher-level features
Block 4	Conv2D 256 + BatchNorm + MaxPool + Dropout 0.35	Complex visual features
Classifier	GlobalAveragePooling2D	Reduce spatial feature maps
Dense	128 neurons + BatchNorm + Dropout 0.50	Learn classification features
Output	1 neuron, Sigmoid	Binary classification probability

Why these components are used

- **Convolution:** extracts visual features such as edges, shapes, and pothole patterns.
- **Max Pooling:** reduces spatial dimensions and computational cost.
- **Batch Normalization:** stabilizes and accelerates training.
- **Dropout:** reduces overfitting by randomly disabling neurons during training.
- **Global Average Pooling:** reduces the number of parameters compared with a large Flatten layer.
- **Sigmoid:** converts the final output into a binary probability.

7. Model Compilation

The CNN is compiled using the Adam optimizer with an initial learning rate of **0.0001**. Binary cross-entropy is used because this is a two-class classification problem.

Parameter	Value
Optimizer	Adam
Initial Learning Rate	0.0001
Loss Function	Binary Cross-Entropy
Metric	Accuracy
Output Activation	Sigmoid

8. Training Strategy

The improved model is trained for a maximum of 50 epochs. The actual number of epochs may be lower because Early Stopping can terminate training when validation performance stops improving.

Callback	Function
EarlyStopping	Stops training when validation loss stops improving and restores the best weights.
ReduceLROnPlateau	Reduces the learning rate when validation loss becomes stagnant.
ModelCheckpoint	Saves the model with the best validation accuracy.

9. Evaluation Methodology

The trained CNN is evaluated using multiple measures rather than relying only on accuracy. The project generates an accuracy graph, loss graph, classification report, confusion matrix, and actual-versus-predicted image visualization.

- **Accuracy:** percentage of correctly classified images.
- **Loss:** measures the difference between predicted probabilities and true labels.
- **Precision:** measures how many predicted positive samples are actually positive.
- **Recall:** measures how many actual positive samples are correctly identified.
- **F1-score:** harmonic mean of precision and recall.
- **Confusion Matrix:** shows correct and incorrect predictions for each class.

10. Previous Training Observation

During an earlier 60-epoch training run, the model achieved a best observed validation accuracy of approximately **96.53%**, while training accuracy reached approximately **97.58%**. Later epochs showed fluctuations in validation performance, indicating some overfitting and motivating the improved training strategy documented in this version.

Important: the 96.53% figure belongs to the earlier training run. The final accuracy of the improved model should be taken from Cell 16 after running the improved code; it should not be assumed in advance.

11. Problems Encountered and Solutions

Problem	Solution
Dataset train folder not found	Added automatic directory/class detection instead of assuming a fixed folder structure.
Kaggle download/path issues	Used a downloadable dataset workflow and verified extracted folders before training.
Accuracy graph not appearing	Separated accuracy and loss plots and ensured the training History object is used.
Validation accuracy fluctuation	Added augmentation, Batch Normalization, Dropout, EarlyStopping and ReduceLROnPlateau
Potential overfitting	Used stronger regularization and best-model checkpointing.

12. Model Files

- **Best checkpoint:** /content/best_road_damage_cnn.keras
- **Final saved model:** /content/road_damage_cnn_improved.keras

The Keras model files can be reused later for inference or integrated into a web/mobile application after adding an appropriate deployment interface.

13. Project Workflow

1. Dataset Download	2. Dataset Extraction
3. Folder/Class Detection	4. Image Preprocessing
5. Data Augmentation	6. Train/Validation Split
7. CNN Model Construction	8. Model Compilation
9. CNN Training	10. Early Stopping / LR Reduction
11. Best Model Saving	12. Evaluation
13. Prediction	

14. Limitations

- The system is a binary classifier and does not locate individual potholes with bounding boxes.
- Performance depends on image quality, lighting, road conditions, and dataset diversity.
- The model should be tested on an independent real-world dataset before production deployment.
- A larger and more diverse dataset can improve robustness.

15. Future Improvements

- Use transfer learning with MobileNetV2, EfficientNet, or ResNet.
- Increase dataset diversity with images from different countries, weather conditions, and road surfaces.
- Add object detection or segmentation to locate potholes rather than only classifying the entire image.
- Deploy the model through a Flask/FastAPI web application.
- Develop a mobile application for real-time road inspection.

• Use GPS and image metadata for road-damage mapping.

16. Conclusion

This project demonstrates the complete development of a deep-learning-based road damage image classification system. A CNN was designed to distinguish between Plain road images and Pothole images. The improved training pipeline uses image augmentation, Batch Normalization, Dropout, Global Average Pooling, Early Stopping, learning-rate scheduling, and model checkpointing to improve generalization and reduce overfitting.

The project provides a strong foundation for automated road-condition analysis. For a production-level system, future work should focus on transfer learning, larger datasets, independent testing, and object detection for precise pothole localization.

17. Author

Name	Shafaat Ullah
Project	Road Damage Detection using CNN
Field	Artificial Intelligence / Machine Learning
Platform	Google Colab

End of Documentation